

Genetic Programming Hyper-Heuristics for the Job Shop Scheduling Problem with Interval Durations: Interpretable Rules that Generalize Across Instance Sizes

Hernán Díaz

Abstract

Dispatching rules remain the method of choice for scheduling under tight time budgets, but hand-crafted rules leave substantial performance unexploited, and the automated design of rules has scarcely been explored for scheduling problems with non-deterministic processing times. We present a genetic programming (GP) hyper-heuristic for the job shop scheduling problem with interval durations (IJSP), in which operation durations are known only to lie within an interval and schedules are evaluated by interval arithmetic. The GP evolves priority expressions over an *interval-aware* terminal set that exposes both the worst-case values and the widths of the uncertain quantities, so that evolved rules can react to uncertainty itself. On the 70 interval versions of Taillard’s benchmark, the best evolved rule attains a mean relative error of 18.5% with a single deterministic constructive pass—improving on the strongest deterministic constructive baseline (Giffler–Thompson with MWKR tie-breaking, 29.4%) by over ten points—and generalizes zero-shot across all seven instance size classes despite being evolved on only four training instances of a single size class (20×15). Embedded in a GRASP-style randomized scheme, the evolved rule converts sampling budget into solution quality (14.1% at best-of-1024). Evolved rules are compact, human-readable expressions, and an automatic configuration study with irace shows the approach is robust to its own hyperparameters, with a preference for *smaller* expression trees at no performance cost. We position the method against fixed rules, active schedule generators, and published metaheuristics, characterizing the computational class in which interpretable evolved rules operate.

1 Introduction

Real manufacturing environments rarely know the exact duration of an operation in advance. The interval job shop scheduling problem (IJSP) models this uncertainty in a minimal, distribution-free way: each processing time is only assumed to lie within a known interval, schedules are built and evaluated with interval arithmetic, and solutions are compared by their worst-case (upper-bound) makespan. Population metaheuristics for the IJSP—artificial bee colony variants and tabu search among them—achieve strong results at the cost of minutes of per-instance search [2, 3, 4].

At the opposite end of the computational spectrum live *dispatching rules*: priority functions that build a schedule in a single constructive pass by repeatedly selecting the next operation to start. Their appeal is threefold—speed, interpretability, and trivial deployment—but hand-crafted rules (SPT, MOR, MWKR and relatives) leave a large quality gap: on the benchmark used in this paper the best fixed rule averages 45.4% relative error, versus 4–10% for metaheuristics.

For *deterministic* scheduling, a rich literature has shown that this gap can be narrowed substantially by evolving rules automatically with genetic programming (GP) [6, 7]. For scheduling under interval uncertainty, however, the automated design of dispatching rules remains, to the best

of our knowledge, unexplored. It is not obvious a priori what an evolved rule should do with uncertainty: ignore it and rank by worst case, hedge against wide intervals, or exploit width information in a structured way. Making this possible requires exposing uncertainty to the rule as first-class attributes.

This paper makes the following contributions:

1. A GP hyper-heuristic for the IJSP whose terminal set is *interval-aware*: alongside worst-case processing time, earliest start and work remaining, rules can read the *widths* of these quantities, enabling uncertainty-sensitive prioritization (Section 4).
2. An empirical study on the 70 interval Taillard instances showing that evolved rules (i) beat every deterministic constructive baseline by a wide margin at equal cost, (ii) generalize zero-shot across instance sizes—evolved on 20×15 , best class 50×15 —and (iii) extend naturally to a randomized multi-start regime with monotone quality gains up to best-of-1024 (Section 6).
3. An analysis of *why* the rules work: an anatomy of the evolved expressions (Section 6.2), an ablation isolating the contribution of the interval-width terminals (Section 6.4), and an executional-robustness study showing that the evolved rule yields schedules whose realized makespan deviates less from its prediction than the baselines’, increasingly so as uncertainty grows (Section 6.7).
4. An automatic configuration study (irace, 200 experiments) of the evolutionary hyperparameters, showing the method is robust to its own knobs: tuning yields only a modest, within-noise gain, from higher selection pressure (Section 6.8).

The remainder of the paper is organized as follows. Section 2 reviews related work. Section 3 formalizes the IJSP. Section 4 presents the GP hyper-heuristic and its interval-aware terminal set. Section 5 details the experimental protocol, and Section 6 the empirical study, including the hyperparameter analysis. Section 7 concludes.

2 Related Work

2.1 Job shop scheduling under interval uncertainty

Interval durations offer a distribution-free uncertainty model requiring only bounds, in contrast to stochastic and fuzzy approaches [15]. Prior IJSP work is dominated by population-based meta-heuristics: a genetic algorithm with a systematic study of interval rankings [1], artificial bee colony variants [2, 3], and a population-based tabu search that currently defines the state of the art [4]. All operate in the minutes-per-instance regime. Fast constructive methods for the IJSP have received far less attention; dispatching rules with interval-aware lexicographic comparisons are the de facto baseline.

2.2 Automated design of dispatching rules

GP hyper-heuristics evolve priority expressions over scheduling attributes, and constitute the standard approach to automated rule design for production scheduling [6, 7]. Evolved rules have been shown to outperform hand-crafted ones across job shop variants, with active research on surrogate fitness, feature selection, and multi-objective formulations. Beyond single rules, a productive line combines several evolved rules into *ensembles* that jointly construct or vote on a solution [10], and recent work explores alternatives to evolutionary search for the same design task—systematic tree

search over the space of expression trees [11]—reporting rules of comparable quality but smaller size and better readability. All of this work targets deterministic (or dynamic-stochastic) settings. Closest to our setting, Karunakaran et al. [8] evolve rules for dynamic job shops with uncertain processing times, exposing summary statistics of observed deviations (an exponential moving average) to the rule; uncertainty there is realized stochastically during simulation, and rules are evaluated on sampled scenarios. A recent survey of learned optimisation for the job shop [9], covering both GP and DRL constructive approaches, records no work on interval-valued processing times. To the best of our knowledge, evolving rules whose *inputs* are interval attributes—and whose fitness is computed by interval arithmetic rather than by sampling realizations—has not been explored before.

2.3 Learned constructive heuristics

Deep reinforcement learning provides an alternative family of learned constructive methods for the JSP [13, 14]; GP rules and DRL policies are increasingly studied as competing learning paradigms for the same construction task, and controlled comparisons between them under common protocols have been identified as a gap in the recent literature [9]. Both families occupy the same computational class at deployment (a single constructive pass); a controlled comparison on the IJSP is beyond the scope of this paper and is the object of ongoing work.

3 The Interval Job Shop Scheduling Problem

An IJSP instance consists of n jobs and m machines. Job j is a chain of m operations $o_{j1} \prec o_{j2} \prec \dots \prec o_{jm}$, each requiring a distinct machine $\mu(o_{jk})$ exclusively and non-preemptively. The duration of operation o is an interval $p_o = [\underline{p}_o, \bar{p}_o]$ with $0 < \underline{p}_o \leq \bar{p}_o$: the only assumption is that the realized duration lies within these bounds.

Schedule construction uses interval arithmetic. For intervals $a = [\underline{a}, \bar{a}]$ and $b = [\underline{b}, \bar{b}]$: $a + b = [\underline{a} + \underline{b}, \bar{a} + \bar{b}]$ and $\max(a, b) = [\max(\underline{a}, \underline{b}), \max(\bar{a}, \bar{b})]$ (component-wise). Given an operation processing order (a permutation with repetition of jobs, decoded left to right), the *semiactive* schedule starts each operation at $s_o = \max(C_{\text{job}}, C_{\text{mach}})$ —the component-wise maximum of the completion intervals of its job predecessor and machine predecessor—and completes at $C_o = s_o + p_o$. The makespan $C_{\max} = \max_j C_{o_{jm}}$ (component-wise maximum) is itself an interval. Since intervals admit no natural total order, schedules are compared with an interval ranking: we use the lexicographic order $\leq_{Lex2}$ (upper bound first, lower bound as tie-break), one of the standard rankings of the IJSP literature and the one found in [1] to yield the most robust schedules; the midpoint ranking $\leq_{MP}$ —equivalent to the possibility-degree method of [5]—underlies the $E[C_{\max}]$ metric in which results are reported. All methods in this paper—the GP rules and every baseline—share one implementation of this decoder and evaluator, so reported differences cannot stem from evaluator discrepancies.

Performance is reported as relative error against the crisp Taillard lower bounds: $RE = (E[C_{\max}] - LB)/LB \times 100$, where $E[C_{\max}]$ is the midpoint of the interval makespan.

4 GP Hyper-Heuristic for the IJSP

4.1 Rule representation

An individual is a priority expression tree. Leaves are drawn from the interval-aware terminal set of Table 1; internal nodes from $\{+, -, \times, \div_p, \min, \max, \text{neg}\}$ with protected division. At each

Table 1: Interval-aware terminal set, with exact definitions. For an eligible operation o of job j on machine m : $p_o = [p_o, \bar{p}_o]$ is its duration interval; C_j and C_m are the (interval) completion times of the last scheduled operation of job j and machine m ; $\mathcal{R}_o$ is the set of operations of j after o ; $\mathcal{E}$ is the current eligible set. Terminals are evaluated on *raw* (unnormalized) values.

Terminal	Definition	Role
PT	$\bar{p}_o$	worst-case processing time
PTW	$\bar{p}_o - p_o$	duration uncertainty
EST	$\bar{e}_o = \max(\bar{C}_j, \bar{C}_m)$	worst-case earliest start
ESTW	$\bar{e}_o - e_o$, with $e_o = \max(C_j, C_m)$	start uncertainty
WKR	$\sum_{q \in \mathcal{R}_o} \bar{p}_q$	worst-case work remaining
WKRW	$\sum_{q \in \mathcal{R}_o} (\bar{p}_q - p_q)$	remaining-work uncertainty
NOR	$ \mathcal{R}_o $	operations remaining
SLACK	$\bar{e}_o - \min_{o' \in \mathcal{E}} \bar{e}_{o'}$	start slack within eligible set
ONE	1	constant

Table 2: Evolution parameters (conventional settings; see Section 6.8 for the sensitivity study).

Population / generations	100 / 50
Initialization	ramped half-and-half, depths 2–4 + {SPT, MWKR}
Selection	tournament, size 4
Crossover / mutation prob.	0.8 / 0.2 (subtree; mutation depth ≤ 3)
Elitism	2
Tree-size cap	40 nodes (violations: fitness ∞)
Fitness	mean RE, deterministic rollout, TA11–TA14
Seeds	{1, 2, 3} (RNG of evolution and environment)

scheduling decision the tree is evaluated (vectorized) over all eligible operations and the operation with *lowest* priority is dispatched, yielding a semiactive schedule. Classic rules are points of this search space—SPT is the tree PT and MWKR is $\text{neg}(WKR)$ —and both are injected as seed individuals.

Protected division is defined as $a \div_p b = a/b$ if $|b| > 10^{-9}$ and a otherwise. Priority ties at dispatch time are broken by eligible set order (lowest job index first), which is deterministic; hence a rule maps each instance to exactly one schedule.

4.2 Evolution

Generational GP with the operators and parameters of Table 2. Initialization is ramped half-and-half over depths 2–4 (grow trees terminate early with probability 0.3 per node), with SPT and MWKR injected as two additional seed individuals. Each generation, the elite passes unchanged and the remainder is produced by tournament selection followed by subtree crossover (a uniformly random node of parent A is replaced by a uniformly random subtree of parent B) or, with the complementary probability, subtree mutation (a uniformly random node is replaced by a fresh grow tree of depth ≤ 3). Offspring exceeding the node cap receive fitness ∞ (bloat control). Fitness of a rule is the mean RE of its deterministic rollout over the four training instances—computed with the same environment, decoder and evaluator used by all baselines, which eliminates evaluator mismatch as a confound.

4.3 Randomized multi-start extension

A deterministic rule produces one schedule. For matched comparisons with sampling-based methods we wrap the evolved rule in GRASP-style randomization: with probability $\varepsilon=0.1$ the dispatched operation is drawn uniformly from the eligible set, otherwise the rule decides; sample 0 remains deterministic. Best-of- N then reports the lexicographically best schedule. This uniform perturbation of a single rule is the simplest member of a broader family of randomized rule-based builders; a richer alternative, orthogonal to our contribution, swaps among a *set* of evolved rules during construction [12].

5 Experimental Setup

Instances. The 70 interval versions of Taillard’s instances (TA1–TA70, seven size classes from 15×15 to 50×20), with crisp Taillard lower bounds as the RE reference. Each interval instance is derived from its crisp counterpart with the generation scheme introduced in [1]: every duration p is replaced by the integer interval $[p - \delta, p + \delta]$, with δ drawn uniformly in $[0, 0.15p]$ (maximum deviation 15%); machine routes are unchanged. The midpoint of every interval thus equals the original duration—the crisp instance is exactly the midpoint scenario of its interval version—which makes the crisp Taillard bounds a meaningful reference for $E[C_{\max}]$. After integer rounding, $\approx 20\%$ of operations end up degenerate (deterministic), and the mean relative width is $\approx 13\%$. The same scheme underlies the benchmark of the published IJSP metaheuristics [1, 2, 3], and the interval Taillard set is the one used in [4], which makes the comparison against those methods an exact one (identical uncertainty model and benchmark).

Data split and exclusions. Rules are evolved on TA11–TA14 (20×15), and TA15–TA20 serve as development set: they never contribute to fitness, but design and configuration decisions (including the irace study of Section 6.8) were selected on their performance, so we flag them as subject to development reuse. The remaining 60 instances are fully unseen by every design decision and constitute the honest generalization test.

Budgets. Population 100, 50 generations: $\sim 20,400$ fitness rollouts (≈ 49 min single-core) per seed; three independent seeds.

Baselines. Fixed rules (SPT, LPT, MOR, MWKR) with interval-aware lexicographic comparison; the Giffler–Thompson active-schedule generator with rule tie-breaks (deterministic and ε -randomized inside the conflict set); published IJSP metaheuristics (fEABC [2], ESABC [3], TS [4]) as yardsticks of the search-based computational class.

6 Results

6.1 Evolution behaviour and seed stability

Three seeds converge to training RE of 15.60%, 15.62% and 15.87%—a spread of 0.27 points—with most of the improvement realized by generation 25–30 (Figure 1). The best evolved rules are structurally recognizable: MWKR corrected by earliest-start and slack $\times$ processing-time terms. For example, the best rule of seed 1 is, after simplification, of the form $-(WKR - f(NOR, PT, PTW)) + EST + SLACK \cdot PT$: maximize work remaining, penalize late earliest starts, and penalize slack weighted by processing time.

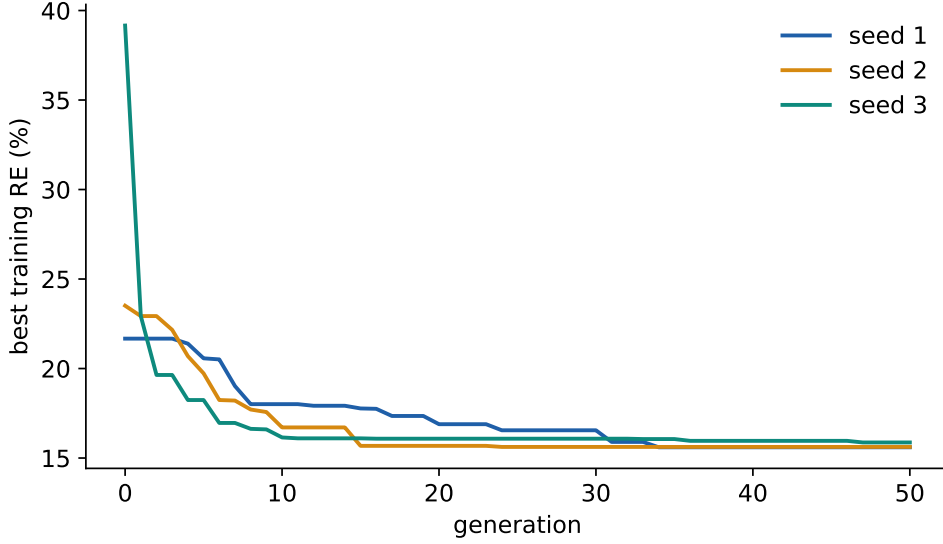


Figure 1: Best training fitness per generation for the three evolution seeds (default configuration).

Table 3: Mean RE (%) per size class, single deterministic rollout, three GP seeds; all 70 Taillard interval instances.

Rule	15×15	20×15	20×20	30×15	30×20	50×15	50×20	All
GP seed 1	16.0	18.7	19.4	20.9	25.0	14.5	15.1	18.5
GP seed 2	16.5	18.4	21.2	21.8	26.9	15.1	16.3	19.5
GP seed 3	18.6	18.5	20.0	22.7	25.7	15.2	15.7	19.5

6.2 Rule anatomy and interpretability

A practical appeal of GP over neural policies is that its output is a readable expression. Evolved rules are compact—mean tree size ≈ 32 nodes, mean depth ≈ 11 —so a rule can be inspected and simplified by hand. Figure 2 reports how often each terminal is used across the evolved rules. The backbone is the classical scheduling triad—work remaining (*WKR*), earliest start (*EST*) and slack (*SLACK*)—together with worst-case processing time (*PT*), confirming that the evolution rediscovers and refines known dispatching logic rather than exploiting benchmark artefacts. Crucially, the *interval-width* terminals (*PTW*, *ESTW*, *WKRW*) account for $\approx 19\%$ of all terminal usage and appear in *every* evolved rule: the evolution consistently reaches for uncertainty information, even though worst-case bounds carry the larger share of the signal. Whether that width information translates into better schedules is quantified by the ablation of Section 6.4 and the robustness analysis of Section 6.7. [TODO: cifras de anatomía sobre el conjunto de 30 semillas tras la campaña]

6.3 Generalization to the full benchmark

Table 3 reports single-rollout RE per size class over all 70 instances. The best rule attains 18.5% global mean—evolved only on 20×15, its best class is the unseen 50×15 (14.5%). Because all terminals are per-operation attributes with no size-bound quantities, transfer requires no retraining or rescaling.

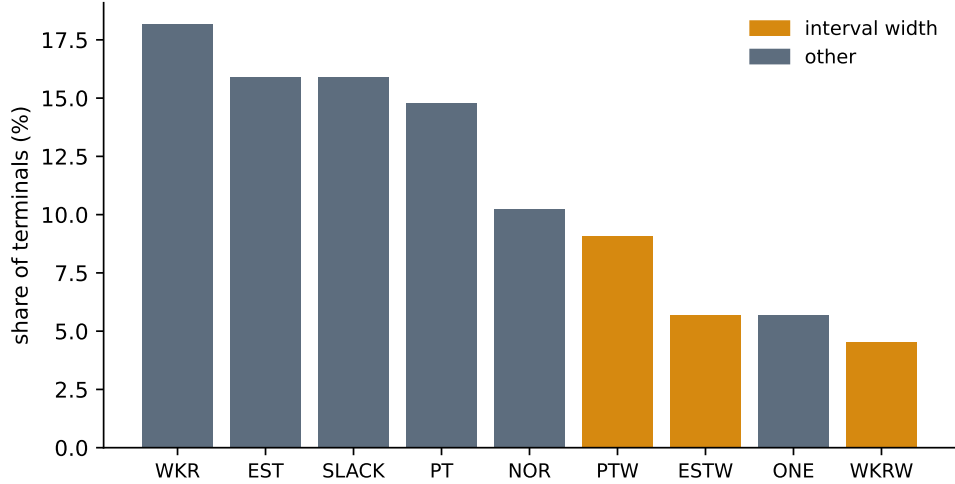


Figure 2: Terminal usage across the evolved rules, as a share of all terminals. Interval-width terminals (amber) expose the magnitude of the uncertainty to the rule.

6.4 The value of interval-awareness: terminal ablation

The interval-width terminals are the distinctive ingredient of this setting, so we isolate their contribution. We re-run the full evolution—30 independent seeds per configuration—with and without the width terminals (*PTW*, *ESTW*, *WKRW*) in the terminal set, everything else held fixed, and compare the resulting mean RE over the 70 instances with a paired Wilcoxon signed-rank test on the shared seeds. [TODO: rellenar con la campaña 30×3: media±std no-width vs full, Δ RE, y el estadístico de Wilcoxon. Redacción según el signo del efecto (paga / cotas de peor caso bastan).]

6.5 Position among constructive methods

All three constructive methods build one schedule per instance in a single pass: MOR averages 45.5% RE, G&T-MWKR 29.5%, and the evolved rule 18.6% (tuned configuration: 17.7%, Section 6.8). The gap is not marginal and not an artefact of averaging: the evolved rule is better than MOR on all 70 instances and than G&T-MWKR on 69 of 70, and a paired Wilcoxon signed-rank test rejects equality against each baseline at $p < 10^{-6}$ ($z = -7.3$). Per-instance figures are given in Table 5 (appendix). Crucially, this quality comes at no evaluation penalty over hand-crafted rules: evaluating the evolved expression tree over the eligible set costs the same, within measurement noise, as evaluating MOR or G&T-MWKR (the three differ by under 15% in per-schedule construction time in our implementation), since all reduce to one vectorized pass over eligible operations per decision. Both cost and quality thus place the evolved rule with the constructive methods, orders of magnitude below the search-based computational class in cost while closing much of the quality gap to it. Figure 3 places these results on the full method ladder, including the published tabu search [4] as the yardstick of the search-based class (an iterative search of minutes per instance versus a single constructive pass).

6.6 Sampling behaviour (GP- ϵ)

Best-of- N over all 70 instances: 18.5% ($N=1$, deterministic), 17.1% (16), 15.9% (64), 14.9% (256), 14.1% (1024)—4.4 points over ten doublings (Figure 4)—closing a substantial part of the gap

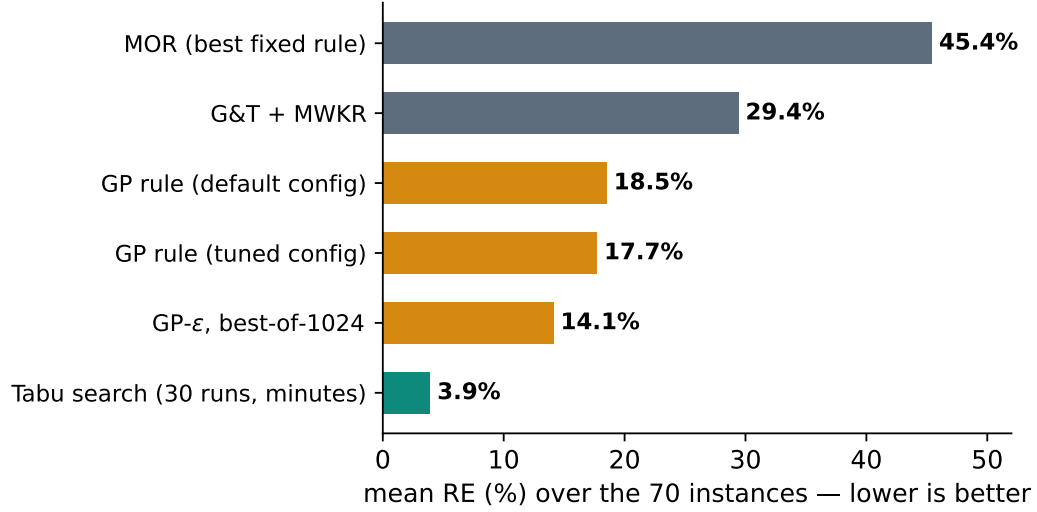


Figure 3: Constructive-method ladder on the 70 interval Taillard instances. Bars are single-solution methods except where noted.

to population-based methods while remaining a set of independent constructive passes, orders of magnitude below the iterative search in computational class and trivially parallel.

6.7 Robustness under executorial uncertainty

Relative error measures a-priori quality, but for the IJSP a schedule is ultimately *executed* under one realization of the durations. A good predictive schedule is one whose executed makespan stays close to its a-priori prediction. We quantify this with the $\bar{\epsilon}$ robustness of [1]: for a schedule with fixed processing order and predicted expected makespan $E[C_{\max}]$ (interval midpoint), we draw $K=1000$ realizations of the durations (uniform on each operation’s interval), execute the fixed order, and average the relative deviation of the executed makespan $C_{\max}^{ex}$ from the prediction, $\bar{\epsilon} = \frac{1}{K} \sum_k |C_{\max}^{ex,k} - E[C_{\max}]| / E[C_{\max}]$. Lower is more robust. To probe behaviour under growing uncertainty we also widen every interval symmetrically about its midpoint by +20% and +40% (clipping negative bounds to zero), re-evaluating the same processing orders.

The evolved rule does not merely produce lower-makespan schedules—it produces *more robust* ones, on two complementary measures (Table 4). First, its predicted makespan interval is the *tightest*: a relative width of 12.6% against 13.8% (G&T-MWKR) and 14.4% (MOR), so the a-priori uncertainty attached to the schedule is smaller. Second, and more tellingly, its executed makespan tracks the prediction most closely: $\bar{\epsilon}$ is lower than for both baselines at the nominal width (5.8 versus 6.5 for MOR and 6.3 for G&T-MWKR), and its advantage *widens* as uncertainty grows (at +40%: 7.7 versus 8.9 and 8.6). Because the rule can read interval widths, it favours dispatching decisions whose realized outcome is less sensitive to how the durations materialize. Figure 5 shows the per-instance $\bar{\epsilon}$ distributions.

[TODO: tras la campaña: rehacer con la mejor regla de 30 semillas y anadir el brazo no-width (robustez interval-aware vs ablacion); test estadistico.]

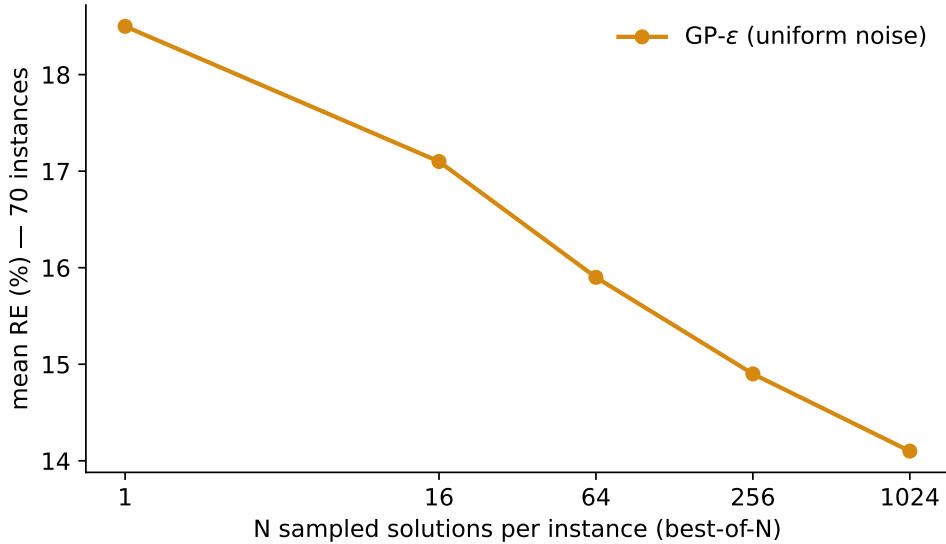


Figure 4: Best-of- N curve of the evolved rule with uniform ε -randomization ($\varepsilon=0.1$): mean RE over the 70 instances as the sampling budget doubles.

Table 4: Robustness over the 70 instances (lower is better on both measures). *Rel. width*: relative width of the predicted makespan interval, $(\bar{C}_{\max} - \underline{C}_{\max})/E[C_{\max}]$ (%). $\bar{\varepsilon} (\times 10^3)$: mean deviation of the executed makespan from its prediction, at nominal interval widths and widened by +20% and +40% (Monte Carlo, $K=1000$ realizations, common random numbers across methods).

Method	Rel. width (%)	$\bar{\varepsilon} (\times 10^3)$		
		+0%	+20%	+40%
GP rule	12.6	5.8	6.8	7.7
G&T-MWKR	13.8	6.3	7.5	8.6
MOR	14.4	6.5	7.7	8.9

6.8 Hyperparameter study

We tune the four qualitative knobs of the evolution (tournament size, crossover probability, tree-size cap, elitism) with irace over a budget of 200 experiments, each a full evolution (population 100, 50 generations) on the training instances evaluated on the development set, with the conventional configuration seeded. The four elite configurations agree on a higher selection pressure than the default (tournament size 7 versus 4) and a crossover probability close to the default (0.72–0.77 versus 0.8); the tree-size cap, by contrast, does not discriminate (surviving elites span caps of 20, 30 and 40), indicating the method is insensitive to bloat control within this range. Throughout the race the statistical tests discard configurations only weakly, itself evidence that performance is largely flat over the hyperparameter space.

Under the same pre-registered confirmation applied to every design decision—the winning configuration re-evolved over three independent seeds and each resulting rule evaluated on all 70 instances—the tuned configuration is *marginally* better than the untuned default: 17.67% versus 18.5% best-of-three-seeds (18.68% versus 19.17% mean). The improvement of 0.5–0.8 points is consistent in direction across both statistics but lies within the seed-to-seed variance (≈ 2 points),

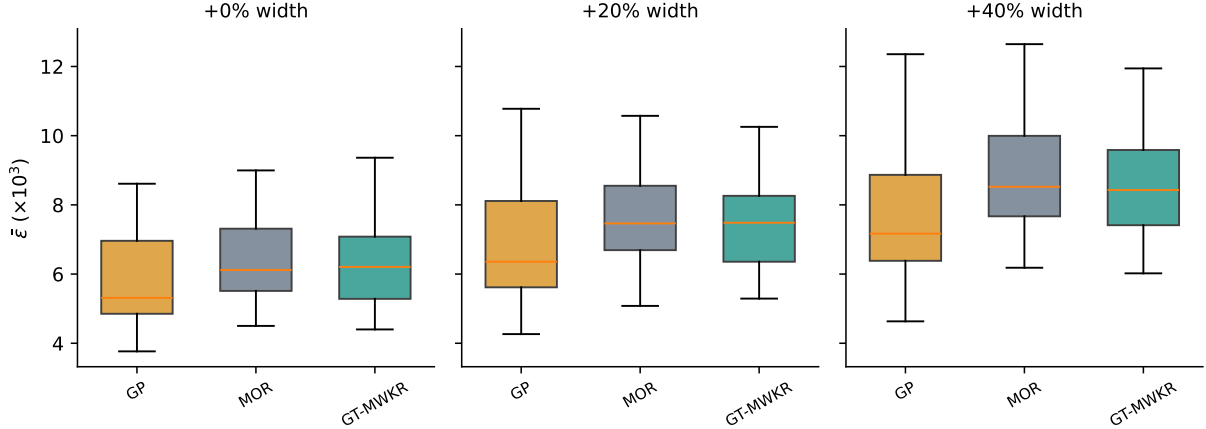


Figure 5: Per-instance $\bar{\epsilon}$ robustness by method at each uncertainty level (lower is better). The evolved rule’s robustness advantage over the constructive baselines grows with interval width.

so we report it as a modest gain rather than a decisive one, and characterize the method as robust to its hyperparameters with a mild benefit from stronger selection pressure.

7 Conclusions and Future Work

We presented what is, to the best of our knowledge, the first GP hyper-heuristic for job shop scheduling with interval durations, built on a terminal set that exposes interval widths as first-class attributes. The empirical conclusions are three. (i) Evolved rules dominate every deterministic constructive baseline on the 70-instance benchmark by a wide, statistically significant margin (18.5% versus 29.4% for the strongest one) at the cost of a single constructive pass, and (ii) they generalize zero-shot across all size classes—evolved on 20×15 , best on the unseen 50×15 —because the representation contains no size-bound quantities. (iii) The method is robust to its evolutionary hyperparameters: a 200-experiment irace study yields only a modest, within-noise gain over conventional settings, from higher selection pressure.

Future work follows three lines. First, richer symbolic models: ensembles of complementary rules [10] and co-evolution of construction and tie-breaking. Second, using evolved rules as cheap, high-quality population initializers for the metaheuristics that define the state of the art in the IJSP, where preliminary pools built from randomized rule rollouts are already compatible with the seeding protocol of [4]. Third, a controlled comparison with learned neural constructive policies under matched training and inference budgets—the head-to-head evaluation that the recent literature [9] identifies as missing.

Reproducibility

The GP hyper-heuristic is implemented from scratch in pure Python/NumPy, with no evolutionary-computation dependencies; evolved rules are stored as plain JSON expression trees and evaluated as drop-in dispatching strategies by the same environment used by every baseline. [\[TODO: URL/DOI del repositorio al publicar\]](#)

Table 5: Per-instance RE (%) of the constructive methods (single deterministic pass). LB is the crisp Taillard lower bound.

Inst.	LB	MOR	G&T	GP	Inst.	LB	MOR	G&T	GP
TA1	1231	29.7	26.1	18.5	TA36	1819	57.4	42.8	23.0
TA2	1244	34.8	27.8	15.7	TA37	1771	43.0	35.6	18.6
TA3	1218	42.9	33.2	18.0	TA38	1673	50.6	28.0	24.6
TA4	1175	62.6	27.7	21.4	TA39	1795	50.2	28.9	16.3
TA5	1224	50.7	33.0	16.4	TA40	1651	42.7	42.3	26.8
TA6	1238	36.6	18.2	12.4	TA41	1906	53.5	45.0	33.5
TA7	1227	39.4	26.1	18.5	TA42	1884	61.3	38.1	25.2
TA8	1217	41.3	25.0	13.6	TA43	1809	61.4	39.8	26.1
TA9	1274	38.2	28.2	15.1	TA44	1948	58.8	34.7	25.2
TA10	1241	38.0	22.1	11.6	TA45	1997	59.5	33.7	20.3
TA11	1357	49.9	30.7	16.2	TA46	1957	59.7	31.6	21.1
TA12	1367	47.8	34.6	17.6	TA47	1807	78.3	30.1	25.2
TA13	1342	46.5	26.9	15.3	TA48	1912	52.2	33.7	23.0
TA14	1345	44.1	29.7	13.3	TA49	1931	46.9	35.3	21.6
TA15	1339	51.3	30.6	21.5	TA50	1833	56.2	46.2	29.1
TA16	1360	35.8	34.0	20.4	TA51	2760	38.2	34.7	23.5
TA17	1462	50.1	17.7	17.9	TA52	2756	38.1	26.5	16.5
TA18	1377	54.2	31.8	26.2	TA53	2717	28.2	18.3	11.0
TA19	1332	43.2	29.3	18.2	TA54	2839	23.6	16.1	7.8
TA20	1348	43.7	24.1	22.2	TA55	2679	37.6	29.2	18.7
TA21	1642	40.8	28.0	19.2	TA56	2781	44.8	19.8	12.3
TA22	1561	46.3	30.8	20.1	TA57	2943	29.9	20.0	11.8
TA23	1518	45.2	35.3	21.9	TA58	2885	30.5	26.3	12.7
TA24	1644	36.2	23.8	15.1	TA59	2655	45.2	28.2	16.8
TA25	1558	53.2	33.8	21.4	TA60	2723	25.3	19.8	14.0
TA26	1591	52.4	31.2	22.7	TA61	2868	48.7	25.4	18.6
TA27	1652	49.8	31.1	16.1	TA62	2869	43.5	27.6	15.5
TA28	1603	38.8	26.8	20.3	TA63	2755	46.0	23.8	14.9
TA29	1583	56.4	32.0	19.1	TA64	2702	43.3	28.4	15.4
TA30	1528	53.3	36.1	19.1	TA65	2725	44.7	30.0	19.6
TA31	1764	34.4	34.4	26.0	TA66	2845	38.1	28.0	12.6
TA32	1774	55.8	33.3	19.7	TA67	2825	58.4	26.1	14.5
TA33	1788	41.6	34.8	32.4	TA68	2784	45.8	19.1	9.6
TA34	1828	34.8	31.8	16.7	TA69	3071	38.0	22.8	14.0
TA35	2007	30.5	24.1	5.5	TA70	2995	52.3	23.4	16.3

A Per-instance results

Table 5 lists, for each of the 70 interval Taillard instances, the crisp lower bound and the single-pass RE (%) of the three deterministic constructive methods.

[TODO: tabla por instancia: se refresca con la mejor regla de la campana 30x3]

References

- [1] H. Díaz, I. González-Rodríguez, J. J. Palacios, I. Díaz, C. R. Vela. A genetic approach to the job shop scheduling problem with interval uncertainty. In *Information Processing and Management of Uncertainty in Knowledge-Based Systems (IPMU)*, pages 663–676. Springer, 2020.
- [2] H. Díaz, J. J. Palacios, I. González-Rodríguez, C. R. Vela. Fast elitist ABC for makespan optimisation in interval JSP. *Natural Computing*, 22(4):645–657, 2023.

- [3] H. Díaz, J. J. Palacios, I. González-Rodríguez, C. R. Vela. An elitist seasonal artificial bee colony algorithm for the interval job shop. *Integrated Computer-Aided Engineering*, 30(3):223–242, 2023.
- [4] H. Díaz et al. Neighbourhood structures and ranking operators for the interval job shop scheduling problem. Under review, 2026.
- [5] D. Lei. Population-based neighborhood search for job shop scheduling with interval processing time. *Computers & Industrial Engineering*, 62(4):1200–1208, 2012.
- [6] J. Branke, S. Nguyen, C. W. Pickardt, M. Zhang. Automated design of production scheduling heuristics: a review. *IEEE Transactions on Evolutionary Computation*, 20(1):110–124, 2016.
- [7] S. Nguyen, Y. Mei, M. Zhang. Genetic programming for production scheduling: a survey with a unified framework. *Complex & Intelligent Systems*, 3(1):41–66, 2017.
- [8] D. Karunakaran, Y. Mei, G. Chen, M. Zhang. Dynamic job shop scheduling under uncertainty using genetic programming. In *Intelligent and Evolutionary Systems*, pages 195–210. Springer, 2017.
- [9] M. Xu, Y. Mei, F. Zhang, M. Zhang. Learn to optimise for job shop scheduling: a survey with comparison between genetic programming and reinforcement learning. *Artificial Intelligence Review*, 58:160, 2025.
- [10] F. J. Gil-Gala, C. Mencía, M. R. Sierra, R. Varela. Learning ensembles of priority rules for online scheduling by hybrid evolutionary algorithms. *Integrated Computer-Aided Engineering*, 28(1):65–80, 2021.
- [11] F. J. Gil-Gala, M. Đurašević, M. R. Sierra, R. Varela. Tree search hyper-heuristic with application to combinatorial optimization. *Journal of Heuristics*, 31:25, 2025.
- [12] F. J. Gil-Gala, M. Đurašević, M. R. Sierra, J. Puente. Greedy randomised adaptive solution builder using priority rules. In *Proc. Genetic and Evolutionary Computation Conference Companion (GECCO '25)*, pages 211–214. ACM, 2025.
- [13] C. Zhang, W. Song, Z. Cao, J. Zhang, P. S. Tan, C. Xu. Learning to dispatch for job shop scheduling via deep reinforcement learning. *NeurIPS*, 2020.
- [14] J. Park, J. Chun, S. H. Kim, Y. Kim, J. Park. Learning to schedule job-shop problems: representation and policy learning using graph neural network and reinforcement learning. *International Journal of Production Research*, 59(11):3360–3377, 2021.
- [15] J. J. Palacios, I. González-Rodríguez, C. R. Vela, J. Puente. Coevolutionary makespan optimisation through different ranking methods for the fuzzy flexible job shop. *Fuzzy Sets and Systems*, 278:81–97, 2015.